Valuing Colombian Financial Instruments with Numerical Methods

First Report

Emmanuel Ruiz
Universidad EAFIT – School of Engineering
Course: Numerical Processes
Instructor: Edwar Samir Posada Murillo

April 2026

Repository: https://github.com/CoSmiC-Ruiz/Procesos-Numericos-Finanzas
Language: Python | Added value: English version, extra methods, LaTeX

Contents

1 Introduction and real-world application

This is the first report for the Numerical Processes final project. The work applies root-finding and linear-system methods to two problems from the Colombian financial market. I picked these problems for two reasons. They show what numerical methods can do when no closed-form solution exists, and they tie the math from the course to actual practice in financial engineering.

Why these problems need numerical methods

A lot of financial problems look simple but cannot be solved by algebra. Computing the yield of a bond, for example, requires isolating a variable that sits as the base of a power with a variable exponent, and that expression has no closed form. Portfolio optimization is different: it does reduce to a linear system, but the size and structure of the system still call for an efficient algorithm.

This report covers two applications:

- **Chapter 1, nonlinear equations:** computing the Yield to Maturity (YTM) of a Colombian TES bond. Eight root-finding methods are tested.
- **Chapter 2, linear systems:** Markowitz portfolio optimization with three stocks from the Colombian Stock Exchange (BVC), and bond sensitivity analysis using the Thomas algorithm.

2 Chapter 1: Root-finding – YTM of a TES bond

2.1 Conceptual background: what is a TES bond?

Financial context: What is a TES?

TES (*Títulos de Tesorería*) are bonds issued by the Colombian national government through the Ministry of Finance and managed by the central bank (Banco de la República). They work like a loan from the investor to the State: the investor pays a certain amount today and, in exchange, receives periodic payments (*coupons*) plus the face value at maturity. Market participants treat TES as low-risk peso-denominated assets because the national government backs them.

Basic bond concepts. A few definitions first:

- **Face value F:** the amount the issuer promises to repay at maturity. The nominal "principal" of the bond.
- **Coupon rate:** the annual percentage of the face value paid periodically to the investor. A 7% coupon on a \$1,000 face value pays \$70 per year.
- **Market price P:** the current price at which the bond trades in the secondary market. It can differ from the face value.
- **Maturity n:** the date when the issuer returns the face value.
- **Yield to Maturity (YTM, r):** the discount rate that makes today's price equal to the present value of all future cash flows. The bond's internal rate of return if held to maturity.

Why bonds trade at a discount or premium. When market rates rise above the coupon rate, investors want a higher return to hold the bond. The coupon cash flows are fixed, so the only way to raise the effective return is to pay less for the bond. When that happens, the bond trades "at a discount" ($P < F$). The opposite case is called "at a premium" ($P > F$).

2.2 Deriving the price equation

The price-yield relationship comes from a basic principle of finance: **the present value of a future cash flow is the cash flow discounted at an appropriate rate**. If an amount C is received in t years and the annual discount rate is r, its present value today is:

$$PV = \frac{C}{(1+r)^t}$$

The reasoning is simple. Investing PV today at rate r would grow to exactly $PV \cdot (1+r)^t = C$ in t years.

A bond produces two kinds of cash flows:

1. Coupons of C at the end of each year $t = 1, 2, \ldots, n$.
2. The face value F at the end of year n.

The fair price of the bond is the present value of all these cash flows:

$$P = \underbrace{\sum_{t=1}^{n} \frac{C}{(1+r)^t}}_{\text{PV of coupons}} + \underbrace{\frac{F}{(1+r)^n}}_{\text{PV of face value}}$$

Given P, finding the effective yield r means solving:

$$f(r) = \sum_{t=1}^{n} \frac{C}{(1+r)^t} + \frac{F}{(1+r)^n} - P = 0 \tag{1}$$

Why r cannot be solved algebraically. The variable r sits in denominators with *different exponents* ($t = 1, 2, \ldots, n$). For $n = 5$ this becomes a degree-5 polynomial in $(1+r)$, but with coefficients that mix C and F. Closed-form formulas exist for polynomials of degree up to 4, but no general algebraic method handles this case, and for $n \geq 5$ the Abel-Ruffini theorem rules out a solution by radicals. So numerical methods are the way forward.

2.3 Problem parameters

The TES bond used here has:

- Annual coupon $C = \$70$ on face value $F = \$1{,}000$ (coupon rate = 7%).
- Maturity $n = 5$ years.
- Market price $P = \$950$.

The equation to solve is:

$$f(r) = \sum_{t=1}^{5} \frac{70}{(1+r)^t} + \frac{1000}{(1+r)^5} - 950 = 0 \tag{2}$$

Existence of a sign change. Before applying any method, a quick check confirms a root exists in a reasonable interval:

$$f(0.05) \approx +136.59 > 0, \qquad f(0.10) \approx -63.72 < 0$$

f is continuous on $(0, \infty)$ and changes sign between 0.05 and 0.10, so the **Intermediate Value Theorem (Bolzano)** guarantees at least one $r^* \in (0.05, 0.10)$ with $f(r^*) = 0$. On top of that, $f'(r) < 0$ for $r > -1$, meaning f is strictly decreasing, so the root is *unique* in this interval.

All methods converge to the same value:

$$\boxed{r^* \approx 0.08260906 \quad \text{(that is, 8.26\% per year)}}$$

Financial reading. Since $r^* \approx 8.26\% > 7\%$ = coupon rate, the bond trades **at a discount** relative to its face value. This matches the \$950 price below the \$1,000 nominal.

Numerical check: with different data

To validate the formulation, the same procedure was repeated for a different TES: $C = \$80$, $F = \$1{,}000$, $n = 7$ years, $P = \$920$. The result was $r^* \approx 9.6229\%$ (coupon $8\% <$ YTM, so the bond trades at a discount, consistent with the setup). Details are in the repository.

Note on Fixed-Point and Steffensen

Both methods need a contractive iteration function $g(x)$, that is, $|g'(x^*)| < 1$ near the root. For equation (2), no natural rearrangement satisfies this. A numerical check of $|g'(r)|$ near $r^* \approx 0.0826$ for several rearrangements (isolating r from the first coupon, damped updates, and so on) gives values greater than 1, confirming divergence. So these two methods use an auxiliary polynomial instead:

$$f(x) = x^3 - x - 2 = 0, \quad g(x) = (x+2)^{1/3}$$

Here $|g'(x^*)| = \frac{1}{3}(x^* + 2)^{-2/3} \approx 0.23 < 1$, so convergence holds. This auxiliary example is unrelated to the TES bond and only demonstrates the methods.

2.4 Bisection method

Theory

Theorem: Bolzano's theorem for bisection

Let $f : [a, b] \to \mathbb{R}$ be a continuous function with $f(a) \cdot f(b) < 0$. Then there is at least one $c \in (a, b)$ such that $f(c) = 0$.

Proof (idea)**.** Since $f(a)$ and $f(b)$ have opposite signs, zero lies in the image $f([a, b])$. By continuity and the Intermediate Value Theorem applied to the value 0, there exists $c \in (a, b)$ with $f(c) = 0$. □

Algorithm. The method takes the midpoint $x_m = (a + b)/2$ and keeps the subinterval whose endpoints still have opposite signs. After k iterations, the interval length is $(b - a)/2^k$, so the error is bounded by:

$$|x_m - r^*| \leq \frac{b-a}{2^{k+1}}$$

Convergence. The method has *linear* convergence with ratio 1/2: each iteration gains one bit of precision. Slow but guaranteed.

Checking the stopping criterion

The number of iterations needed to reach a tolerance ε is:

$$k \geq \log_2\left(\frac{b-a}{\varepsilon}\right) = \log_2\left(\frac{0.10 - 0.05}{10^{-7}}\right) \approx 18.93$$

that is, $k = 19$. As shown below, the method indeed converges in exactly 19 iterations, matching the theoretical bound.

Results – TES, $a = 0.05$, $b = 0.10$, tol $= 10^{-7}$

Table 1: Bisection – selected iterations. Error criterion: $E = |x_m - x_m^{\text{prev}}|$.

Iter	a	x_m	b	$f(x_m)$	E
1	0.05000000	0.07500000	0.10000000	$+2.98 \times 10^{1}$	—
2	0.07500000	0.08750000	0.10000000	-1.85×10^{1}	1.25×10^{-2}
3	0.07500000	0.08125000	0.08750000	$+5.23 \times 10^{0}$	6.25×10^{-3}
4	0.08125000	0.08437500	0.08750000	-6.74×10^{0}	3.13×10^{-3}
5	0.08125000	0.08281250	0.08437500	-7.80×10^{-1}	1.56×10^{-3}
6	0.08125000	0.08203125	0.08281250	$+2.22 \times 10^{0}$	7.81×10^{-4}
			$\vdots$		
19	0.08260870	0.08260908	0.08260947	≈ 0	$< 10^{-7}$

Result: $r^* \approx 0.08260908$ in 19 iterations.

Method analysis

The observed behavior matches the theory exactly: the error E is roughly halved at each iteration (from 1.25×10^{-2} to 6.25×10^{-3}, then 3.13×10^{-3}, and so on), as predicted by linear convergence with ratio 1/2. This regular pattern is what makes bisection so easy to recognize: predictable, robust, slow. In applications where each evaluation of f is expensive, like Monte Carlo pricing of exotic bonds, 19 evaluations may be too many, and higher-order methods are preferred. When f is cheap and robustness matters more, bisection still works fine. Automated rate calibration in production systems is one example.

2.5 False position (Regula Falsi)

Theory

Instead of the midpoint, false position uses the *x-intercept of the chord* connecting $(a, f(a))$ and $(b, f(b))$:

$$x_r \;=\; b \;-\; f(b)\,\frac{b-a}{f(b)-f(a)}$$

Geometric justification. The line through $(a, f(a))$ and $(b, f(b))$ has equation:

$$y - f(a) = \frac{f(b)-f(a)}{b-a}(x-a)$$

Setting $y = 0$ and solving for x yields the formula above. If f were linear, x_r would be the exact root.

Updating the interval. Same as in bisection: the subinterval where the sign change persists is kept. Convergence is *superlinear* when f is reasonably convex or concave, but can degrade to linear if one endpoint of the bracket stays fixed (a known issue called *stagnation*).

Results – TES, $a = 0.05$, $b = 0.10$, tol $= 10^{-7}$

Table 2: False position – all iterations. $E = |x_r - x_r^{\text{prev}}|$.

Iter	a	x_r	b	$f(x_r)$	E
1	0.05000000	0.08409400	0.10000000	-5.67×10^{0}	—
2	0.05000000	0.08273484	0.08409400	-4.82×10^{-1}	1.36×10^{-3}
3	0.05000000	0.08261970	0.08273484	-4.08×10^{-2}	1.15×10^{-4}
4	0.05000000	0.08260996	0.08261970	-3.46×10^{-3}	9.75×10^{-6}
5	0.05000000	0.08260913	0.08260996	-2.93×10^{-4}	8.25×10^{-7}
6	0.05000000	0.08260906	0.08260913	-2.48×10^{-5}	6.98×10^{-8}

Result: $r^* \approx 0.08260906$ in 6 iterations.

Method analysis

False position converges much faster than bisection (6 vs. 19 iterations) because it uses the *values* of f, not only its signs. A classic feature of the method also shows up here: the left endpoint $a = 0.05$ **never updates**. The reason is that f is convex on the interval and the chord always sits below the curve, so the zero of the chord always lands on the same side of the true root. The method then stops behaving as a symmetric geometric contraction.

In practice this stagnation does not block convergence but slows it down asymptotically to linear. Modified variants like Illinois or Pegasus detect the stuck endpoint and halve f there, getting back to superlinear convergence. I did not implement them here because the classical version already converges fast enough for the problem.

2.6 Newton-Raphson method

Theory

Newton-Raphson uses the **first-order Taylor expansion** of f around the current iterate x_n:

$$f(x) \approx f(x_n) + f'(x_n)(x - x_n)$$

Setting the right-hand side to zero (the linear approximation's root) gives:

$$0 = f(x_n) + f'(x_n)(x_{n+1} - x_n) \quad \Longrightarrow \quad \boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}}$$

Theorem: Quadratic convergence of Newton

Let f be twice continuously differentiable in a neighborhood of x^* with $f(x^*) = 0$ and $f'(x^*) \neq 0$. Then there exists $\delta > 0$ such that for any $x_0 \in (x^* - \delta, x^* + \delta)$, the sequence $\{x_n\}$ converges to x^* and satisfies:

$$\lim_{n \to \infty} \frac{|x_{n+1} - x^*|}{|x_n - x^*|^2} = \left| \frac{f''(x^*)}{2f'(x^*)} \right|$$

that is, convergence is of order $p = 2$ (quadratic).

What quadratic convergence means. At each iteration the error is squared (times a constant). If $e_n \approx 10^{-3}$, then $e_{n+1} \approx 10^{-6}$, $e_{n+2} \approx 10^{-12}$, and so on. The number of *correct digits doubles* at each step near the root.

Derivative for the TES bond. For f from equation (2):

$$f'(r) = -\sum_{t=1}^{5} \frac{70 \cdot t}{(1+r)^{t+1}} - \frac{5 \cdot 1000}{(1+r)^6}$$

This derivative is negative for $r > -1$, so f is strictly decreasing and the root is unique.

Choice of starting point. The starting point was $x_0 = 0.08$, close to the expected root. Newton does not need a bracket, but a good initial guess is essential: starting too far away can cause divergence or oscillation.

Results – TES, $x_0 = 0.08$, tol $= 10^{-7}$

Table 3: Newton-Raphson. Criterion: $E = |x_{n+1} - x_n| < 10^{-7}$.

Iter	x_i	$f(x_i)$	E
0	0.0800000000	$+1.01 \times 10^{1}$	—
1	0.0825911147	$+6.88 \times 10^{-2}$	2.59×10^{-3}
2	0.0826090542	$+3.26 \times 10^{-6}$	1.79×10^{-5}
3	0.0826090551	≈ 0	8.51×10^{-10}

Result: $r^* \approx 0.08260906$ in 3 iterations.

Method analysis and quadratic convergence check

The data confirm the quadratic convergence the theorem predicts. The error progression is:

$$E_1 = 2.59 \times 10^{-3} \quad \rightarrow \quad E_2 = 1.79 \times 10^{-5} \quad \rightarrow \quad E_3 = 8.51 \times 10^{-10}$$

The number of exact digits goes from about 3 to about 5 to about 10, doubling at each step. That doubling is what order $p = 2$ looks like in practice. The theoretical constant can be checked too:

$$\frac{E_2}{E_1^2} = \frac{1.79 \times 10^{-5}}{(2.59 \times 10^{-3})^2} \approx 2.67$$

This ratio should stabilize near $|f''(x^*)/(2f'(x^*))|$, which confirms the quadratic convergence.

The main practical limitation is that Newton needs f' *analytically*. For the TES bond this is trivial because the derivative comes out term by term. In more complex models like bonds with embedded options, manual differentiation can be costly or impossible, and that is why Secant exists as an alternative.

2.7 Secant method

Theory

Secant replaces the exact derivative in Newton with a **finite difference approximation**:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}$$

Substituting into Newton's formula:

$$\boxed{x_{n+1} = x_n - f(x_n)\,\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}}$$

Geometrically, x_{n+1} is the x-intercept of the *secant line* through the last two points.

Theorem: Convergence order of Secant

Under the same hypotheses as Newton, the secant method converges with order:

$$p = \frac{1+\sqrt{5}}{2} \approx 1.618 \quad \text{(the golden ratio } \varphi\text{)}$$

Comparison with Newton. Although Newton ($p = 2$) converges faster per iteration, Secant *needs only one evaluation of f per iteration* (after the first), while Newton needs f and f'. In total cost, Secant can be more efficient if computing f' is more expensive than computing f.

Results – TES, $x_0 = 0.05$, $x_1 = 0.10$, tol $= 10^{-7}$

Table 4: Secant method. $E = |x_{n+1} - x_n| < 10^{-7}$.

Iter	x_i	$f(x_i)$	E
0	0.0500000000	$+1.37 \times 10^{2}$	—
1	0.1000000000	-6.37×10^{1}	—
2	0.0840940030	-5.67×10^{0}	1.59×10^{-2}
3	0.0825401128	$+2.64 \times 10^{-1}$	1.55×10^{-3}
4	0.0826093258	-1.04×10^{-3}	6.92×10^{-5}
5	0.0826090551	-1.89×10^{-7}	2.71×10^{-7}
6	0.0826090551	≈ 0	4.93×10^{-11}

Result: $r^* \approx 0.08260906$ in 6 iterations, no derivative required.

Method analysis

Convergence is clearly superlinear but not quadratic. Successive errors $1.59 \times 10^{-2} \to 1.55 \times 10^{-3} \to 6.92 \times 10^{-5}$ improve by a factor near $10^{-1.5}$ per step, which lines up with order $p \approx 1.618$. Secant reaches precision similar to Newton (6 vs. 3 iterations) without an analytic derivative. That makes it the preferred option when f is a black box or when f' is awkward, for example in bond pricing functions calibrated with spline curves.

2.8 Fixed-Point iteration (additional method)

Theory

The idea is to rewrite $f(x) = 0$ as a *fixed-point problem* $x = g(x)$, where g is built by isolating x in some way. The iteration is:

$$x_{n+1} = g(x_n)$$

Theorem: Banach Fixed-Point Theorem

Let $g : [a, b] \to [a, b]$ be continuous and suppose there exists $k \in [0, 1)$ with

$$|g(x) - g(y)| \leq k\,|x - y| \quad \forall\, x, y \in [a, b]$$

(that is, g is a *contraction*). Then:

1. There exists a unique fixed point $x^* \in [a, b]$ with $g(x^*) = x^*$.
2. For any $x_0 \in [a, b]$, the sequence $x_{n+1} = g(x_n)$ converges to x^*.
3. The error bound $|x_n - x^*| \leq \dfrac{k^n}{1-k}|x_1 - x_0|$ holds.

Practical condition. If g is differentiable, the contraction condition reduces to $|g'(x)| \leq k < 1$ on the interval. Locally, this means $|g'(x^*)| < 1$ near the root.

Checking the theorem on the auxiliary example

For $f(x) = x^3 - x - 2$, define:

$$g(x) = (x + 2)^{1/3} \quad \Longrightarrow \quad g'(x) = \tfrac{1}{3}(x + 2)^{-2/3}$$

On the interval $[1, 2]$:

$$\max_{x \in [1,2]} |g'(x)| = |g'(1)| = \tfrac{1}{3}(3)^{-2/3} \approx 0.1603 < 1$$

Numerical check: of Banach's theorem

Numerical values of $|g'(x)|$ on a grid of $[1, 2]$:

x	$g(x)$	$\lvert g'(x)\rvert$
1.0	1.442250	0.160250
1.2	1.473613	0.153501
1.4	1.503695	0.147421
1.6	1.532619	0.141909
1.8	1.560491	0.136885
2.0	1.587401	0.132283

The maximum is $k \approx 0.1603 < 1$, and $g([1, 2]) \subseteq [1.44, 1.59] \subset [1, 2]$, so both hypotheses of the theorem hold: g is a contraction of $[1, 2]$ into itself. Convergence is guaranteed.

Results – $f(x) = x^3 - x - 2$, $x_0 = 1.0$, **tol** $= 10^{-7}$

Table 5: Fixed-Point iteration. $E = |x_{n+1} - x_n|$.

Iter	x_i	$f(x_i)$	E
0	1.0000000000	-2.0000×10^{0}	—
1	1.4422495703	-4.4225×10^{-1}	4.42×10^{-1}
2	1.5098974493	-6.7648×10^{-2}	6.76×10^{-2}
3	1.5197243050	-9.8269×10^{-3}	9.83×10^{-3}
4	1.5211412691	-1.4170×10^{-3}	1.42×10^{-3}
5	1.5213453678	-2.0410×10^{-4}	2.04×10^{-4}
6	1.5213747615	-2.9394×10^{-5}	2.94×10^{-5}
7	1.5213789946	-4.2331×10^{-6}	4.23×10^{-6}
8	1.5213796042	-6.0963×10^{-7}	6.10×10^{-7}
9	1.5213796920	-8.7795×10^{-8}	8.78×10^{-8}

Result: $x^* \approx 1.52137969$ in 9 iterations.

Method analysis

The ratio of consecutive errors is very stable:

$$\frac{E_2}{E_1} = \frac{6.76 \times 10^{-2}}{4.42 \times 10^{-1}} \approx 0.153, \quad \frac{E_3}{E_2} \approx 0.145, \quad \frac{E_8}{E_7} \approx 0.144$$

This ratio converges to $|g'(x^*)| \approx 0.144$, confirming numerically the *linear convergence with rate* k the theorem predicts. The iteration count is higher than Newton's, but the method has the advantage of needing no derivative and no bracket. It is also useful when g comes naturally from the problem, like in the implied-volatility equations from Black-Scholes.

2.9 Steffensen's method (additional method)

Theory

Steffensen applies **Aitken's Δ^2 acceleration** to a fixed-point sequence. Given a linearly convergent iteration $x_{n+1} = g(x_n)$, Aitken proposed the extrapolation:

$$\hat{x}_n = x_n - \frac{(x_{n+1} - x_n)^2}{x_{n+2} - 2x_{n+1} + x_n}$$

Derivation. If $\{x_n\}$ converges linearly with rate λ, then $x_{n+1} - x^* \approx \lambda(x_n - x^*)$. Manipulating this relation algebraically, eliminating λ, and solving for x^* yields the extrapolation formula. Steffensen iteratively computes:

$$x_1 = g(x_0), \quad x_2 = g(x_1), \quad x_{\text{new}} = x_0 - \frac{(x_1 - x_0)^2}{x_2 - 2x_1 + x_0}$$

and restarts with $x_0 \leftarrow x_{\text{new}}$.

Theorem: Quadratic convergence of Steffensen

If g is a contraction near x^* with $g'(x^*) \neq 1$, then Steffensen's method converges quadratically to x^* (order $p = 2$), without computing f'.

So Steffensen "costs" the same as a fixed-point iteration (no derivative needed) but reaches the speed of Newton.

Results – same auxiliary example

Table 6: Steffensen. $E = |x_{\text{new}} - x|$.

Iter	x_i	$f(x_i)$	E
0	1.0000000000	-2.0000×10^{0}	—
1	1.5221137197	$+4.3653 \times 10^{-3}$	5.22×10^{-1}
2	1.5213797080	$+7.3432 \times 10^{-9}$	7.34×10^{-4}
3	1.5213797068	≈ 0	1.24×10^{-9}

Result: $x^* \approx 1.52137971$ in 3 iterations.

Method analysis

Compared with Fixed-Point (9 iterations) on the same problem, Steffensen cuts the count by a factor of 3, which fits the jump from order 1 to order 2. The error progression is clearly quadratic: $5.22 \times 10^{-1} \to 7.34 \times 10^{-4} \to 1.24 \times 10^{-9}$, with correct digits going from 0 to 3 to 9, roughly *doubling*. Steffensen is attractive when a natural iteration g exists but its linear convergence is too slow. The limitation, mentioned earlier, is the need for a contractive g, which is not always available, as in the TES bond.

2.10 Müller's method (additional method)

Theory

Müller generalizes the secant method using a **parabola** (rather than a line) through the three most recent points $(x_0, f(x_0))$, $(x_1, f(x_1))$, $(x_2, f(x_2))$. The parabola is built with *Newton divided differences*:

$$p(x) = f(x_2) + (x - x_2)f[x_2, x_1] + (x - x_2)(x - x_1)f[x_2, x_1, x_0]$$

where $f[\cdot, \cdot]$ and $f[\cdot, \cdot, \cdot]$ are first- and second-order divided differences. Expanding, $p(x) = a(x - x_2)^2 + b(x - x_2) + c$.

The next iterate x_3 is found as a root of p via the quadratic formula, choosing the sign that gives the denominator of larger magnitude (this avoids catastrophic cancellation):

$$x_3 = x_2 - \frac{2c}{b + \operatorname{sgn}(b)\sqrt{b^2 - 4ac}}$$

Theorem: Convergence order of Müller

Under standard regularity and simple-root assumptions, Müller's method converges with order:

$$p \approx 1.839 \quad \text{(real root of } p^3 - p^2 - p - 1 = 0\text{)}$$

Unique feature. When $b^2 - 4ac < 0$, the root is complex: Müller can find **complex roots** starting from real initial guesses, which Newton or Secant cannot do without modification. For the TES bond only the real root is of interest.

Results – TES, $x_0 = 0.05$, $x_1 = 0.08$, $x_2 = 0.10$, **tol** $= 10^{-7}$

Table 7: Müller's method. $E = |x_3 - x_2|$.

Iter	x_i	$f(x_i)$	E
0	0.0500000000	$+1.37 \times 10^{2}$	—
1	0.0800000000	$+1.01 \times 10^{1}$	—
2	0.1000000000	-6.37×10^{1}	—
3	0.0826005633	$+3.26 \times 10^{-2}$	1.74×10^{-2}
4	0.0826090530	$+8.00 \times 10^{-6}$	8.49×10^{-6}
5	0.0826090551	$+6.03 \times 10^{-12}$	2.09×10^{-9}

Result: $r^* \approx 0.08260906$ in 5 iterations.

Method analysis

The parabolic approximation captures the curvature of f near the root better than a line does, so convergence is faster than Secant (5 vs. 6 iterations). The empirical ratio of consecutive errors is:

$$\frac{E_2}{E_1^{1.84}} = \frac{8.49 \times 10^{-6}}{(1.74 \times 10^{-2})^{1.84}} \approx 1.5 \cdot 10^{-2}$$

which fits an order near 1.84. Müller's main advantage in finance is the ability to find complex roots: it shows up in problems like the internal rate of return (IRR) for projects with cash flows that change sign, where complex or multiple roots can exist.

2.11 Trisection method (additional method)

Theory

Trisection splits the interval $[a, b]$ into *three* equal subintervals at each iteration, using two interior points:

$$x_1 = a + \frac{b-a}{3}, \qquad x_2 = a + \frac{2(b-a)}{3}$$

The subinterval containing the root is chosen by sign tests:

- If $f(a)\, f(x_1) < 0 \Rightarrow$ root in $[a, x_1]$.
- Else if $f(x_1)\, f(x_2) < 0 \Rightarrow$ root in $[x_1, x_2]$.
- Otherwise $\Rightarrow$ root in $[x_2, b]$.

Theorem: Convergence of trisection

Under the same hypotheses as Bolzano's theorem, trisection converges linearly with ratio $1/3$ per iteration:

$$|x_k - r^*| \leq \frac{b-a}{3^k}$$

Comparison with bisection. Each trisection step shrinks the bracket by $1/3$ vs. $1/2$ for bisection. Fewer iterations are needed for the same tolerance:

$$k_{\text{tri}} = \frac{\log((b-a)/\varepsilon)}{\log 3} \approx 12.1 \quad \to \quad k = 13$$

vs. $k_{\text{bis}} = 19$. **However**, each iteration uses *two* evaluations of f (at x_1 and x_2), not one. Total cost:

$$\text{Trisection evals} = 13 \times 2 = 26 \quad \text{vs.} \quad \text{Bisection evals} = 19$$

Bisection is therefore more efficient *in total evaluations* when f is expensive. Trisection is only competitive when the two evaluations can run in parallel.

Results – TES, $a = 0.05$, $b = 0.10$, tol $= 10^{-7}$

Table 8: Trisection – selected iterations. $E = b - a$.

Iter	a	x_1	x_2	$E = b - a$
1	0.05000000	0.06666667	0.08333333	5.00×10^{-2}
2	0.06666667	0.07222222	0.07777778	1.67×10^{-2}
3	0.07777778	0.07962963	0.08148148	5.56×10^{-3}
4	0.08148148	0.08209877	0.08271605	1.85×10^{-3}
5	0.08209877	0.08230453	0.08251029	6.17×10^{-4}
6	0.08251029	0.08257888	0.08264746	2.06×10^{-4}
7	0.08257888	0.08260174	0.08262460	6.86×10^{-5}
8	0.08260174	0.08260936	0.08261698	2.29×10^{-5}
		$\vdots$		
13	0.08260899	—	—	$< 10^{-7}$

Result: $r^* \approx 0.08260903$ in 13 iterations.

Method analysis

The bracket width shrinks by a factor of 1/3 at each step, just as the theorem predicts, going from 5×10^{-2} to 1.67×10^{-2} to 5.56×10^{-3}, and so on. Final precision is guaranteed by the bracket: the true root sits inside $[0.08260899, 0.08260908]$, matching what the other methods found. As mentioned above, fewer iterations do not automatically mean lower computational cost. Trisection does double work per step.

2.12 Summary of Chapter 1

Table 9: Comparison of all root-finding methods. Results marked with * use the auxiliary polynomial $f(x) = x^3 - x - 2$.

Method	**Iter.**	**Inputs**	**Approximate root**
Bisection	19	f, bracket	$r^* \approx 0.08260908$
False Position	6	f, bracket	$r^* \approx 0.08260906$
Newton-Raphson	3	f, f'	$r^* \approx 0.08260906$
Secant	6	f (2 points)	$r^* \approx 0.08260906$
Müller	5	f (3 points)	$r^* \approx 0.08260906$
Fixed-Point*	9	f, g	$x^* \approx 1.52137969$
Steffensen*	3	f, g	$x^* \approx 1.52137971$
Trisection	13	f, bracket	$r^* \approx 0.08260903$

All methods applied to the TES bond converge to $r^* \approx 8.26\%$. **Newton-Raphson and Steffensen** reach this precision in only 3 iterations because they are quadratically convergent. Iter-

ation count alone does not decide total cost, though. Each evaluation of f (or f') has a price, and the best method depends on the balance between iteration count, cost per iteration, and whether the derivative is available.

3 Chapter 2: Linear systems – Portfolio optimization

3.1 Conceptual background: what is Markowitz portfolio theory?

Financial context: The Markowitz problem

In 1952, Harry Markowitz published *Portfolio Selection* and changed the foundations of modern financial theory. He received the 1990 Nobel Prize in Economics for this work. The main idea is that a rational investor should not pick individual assets based on expected return alone, but **combinations** of assets that give the best trade-off between expected return and risk (measured by variance).

Required background

Expected return. The expected return of asset i is $\mu_i = E[r_i]$. If weights $x_1, \ldots, x_n$ are placed on n assets (with $\sum_i x_i = 1$), the portfolio's expected return is linear:

$$\mu_P = \sum_{i=1}^{n} x_i \mu_i = \mathbf{x}^\top \boldsymbol{\mu}$$

Portfolio variance. Risk is measured by variance, which is *not* linear in the weights because it involves covariances between assets:

$$\sigma_P^2 = \sum_{i=1}^{n} \sum_{j=1}^{n} x_i x_j \sigma_{ij} = \mathbf{x}^\top \Sigma \, \mathbf{x}$$

where Σ is the covariance matrix (symmetric, positive semidefinite).

Why diversification reduces risk. If two assets are not perfectly correlated ($\sigma_{ij} < \sigma_i \sigma_j$), combining them lets the moves of one partly offset the moves of the other. The total variance of the portfolio drops without sacrificing expected return. That is the mathematical reason behind the old saying "don't put all your eggs in one basket".

Problem formulation

The **Markowitz problem** is to find the weights $\mathbf{x} = (x_1, \ldots, x_n)$ that *minimize variance* subject to:

1. Reaching a target expected return R.
2. Being fully invested (budget = 1).

Formally:

$$\min_{\mathbf{x}} \ \tfrac{1}{2}\mathbf{x}^\top \Sigma \, \mathbf{x} \quad \text{subject to} \quad \boldsymbol{\mu}^\top \mathbf{x} = R, \quad \mathbf{1}^\top \mathbf{x} = 1$$

Deriving the KKT system

To solve the problem, form the **Lagrangian** with multipliers λ_1, λ_2 for each constraint:

$$\mathcal{L}(\mathbf{x}, \lambda_1, \lambda_2) = \tfrac{1}{2}\mathbf{x}^\top \Sigma \mathbf{x} - \lambda_1(\boldsymbol{\mu}^\top \mathbf{x} - R) - \lambda_2(\mathbf{1}^\top \mathbf{x} - 1)$$

The first-order **Karush-Kuhn-Tucker (KKT) conditions** are:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \Sigma \mathbf{x} - \lambda_1 \boldsymbol{\mu} - \lambda_2 \mathbf{1} = \mathbf{0} \tag{3}$$

$$\frac{\partial \mathcal{L}}{\partial \lambda_1} = \boldsymbol{\mu}^\top \mathbf{x} - R = 0 \tag{4}$$

$$\frac{\partial \mathcal{L}}{\partial \lambda_2} = \mathbf{1}^\top \mathbf{x} - 1 = 0 \tag{5}$$

Theorem: KKT conditions for QP with linear constraints

Let Σ be symmetric positive definite. Then the quadratic optimization problem with linear constraints has a *unique* global solution that satisfies conditions (3)–(5). Since the Lagrangian is convex, these conditions are both necessary and sufficient.

Rewriting in matrix form $A\mathbf{z} = \mathbf{b}$, with $\mathbf{z} = (x_1, x_2, x_3, \lambda_1, \lambda_2)^\top$:

$$\underbrace{\begin{pmatrix} \Sigma_{11} & \Sigma_{12} & \Sigma_{13} & -\mu_1 & -1 \\ \Sigma_{21} & \Sigma_{22} & \Sigma_{23} & -\mu_2 & -1 \\ \Sigma_{31} & \Sigma_{32} & \Sigma_{33} & -\mu_3 & -1 \\ \mu_1 & \mu_2 & \mu_3 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{pmatrix}}_{A} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \lambda_1 \\ \lambda_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ R \\ 1 \end{pmatrix}$$

So **a quadratic optimization problem reduces to a linear system** of $n + 2$ equations. That is why the course methods (Gauss and its variants) apply directly.

3.2 Problem 1: Markowitz portfolio with three BVC stocks

Assets. Three liquid stocks from the Colombian Stock Exchange:

- **Ecopetrol (ECO):** Colombia's main oil company, sensitive to crude oil prices.
- **Interconexión Eléctrica S.A. (ISA):** energy infrastructure company with relatively stable cash flows.
- **Bancolombia (PFBCOL):** the largest private bank in Colombia, exposed to the credit cycle.

Parameters (illustrative values).

- Annual expected returns: $\mu_1 = 15\%$, $\mu_2 = 12\%$, $\mu_3 = 10\%$.
- Target return: $R = 12\%$.
- Covariance matrix:

$$\Sigma = \begin{pmatrix} 0.0400 & 0.0120 & 0.0080 \\ 0.0120 & 0.0225 & 0.0090 \\ 0.0080 & 0.0090 & 0.0196 \end{pmatrix}$$

Note: The covariance values are assumed for illustration and do not correspond to a specific historical period or data source.

The linear system to solve is:

$$\begin{pmatrix} 0.0400 & 0.0120 & 0.0080 & -0.15 & -1 \\ 0.0120 & 0.0225 & 0.0090 & -0.12 & -1 \\ 0.0080 & 0.0090 & 0.0196 & -0.10 & -1 \\ 0.15 & 0.12 & 0.10 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \lambda_1 \\ \lambda_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0.12 \\ 1 \end{pmatrix}$$

3.3 Problem 2: Bond sensitivity (tridiagonal system)

Context. Bond *duration* and *convexity* measure sensitivity to changes in interest rates. For bond portfolios with several maturities, the effect of *parallel shifts* of the yield curve is modeled with a boundary value problem (BVP):

$$-u''(x) = 1, \quad x \in (0,1), \quad u(0) = u(1) = 0$$

where $u(x)$ represents the accumulated sensitivity. The interval is discretized with $n = 4$ interior nodes and $h = 0.2$. The centered finite-difference approximation is:

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = 1, \quad i = 1,2,3,4, \quad u_0 = u_5 = 0$$

The result is a symmetric tridiagonal system with:

$$\mathbf{b} = (50,50,50,50), \quad \mathbf{a} = \mathbf{c} = (-25,-25,-25), \quad \mathbf{d} = (1,1,1,1)$$

The analytical solution of the continuous problem is $u(x) = x(1-x)/2$, which provides a benchmark.

3.4 Simple Gaussian Elimination

Theory

Given a non-singular matrix A and a vector $\mathbf{b}$, the method transforms $A\mathbf{x} = \mathbf{b}$ into an upper triangular system $U\mathbf{x} = \mathbf{b}'$ using elementary row operations, and then solves it by *back substitution*.

Theorem: Existence of LU without pivoting

Let A be an $n \times n$ matrix. Gaussian elimination without row interchanges produces a factorization $A = LU$ if and only if all leading principal minors of A are nonzero:

$$\det(A_k) \neq 0, \quad k = 1, \ldots, n$$

where A_k is the top-left $k \times k$ submatrix.

If any intermediate pivot becomes zero, the algorithm fails. For ill-conditioned systems, it is also prone to round-off errors.

Computational cost. The elimination requires $\mathcal{O}(n^3/3)$ operations, and back substitution $\mathcal{O}(n^2)$. For large systems, specialized methods (Thomas, Cholesky) are preferred.

Results – 5×5 Markowitz system

Table 10: Simple Gaussian elimination – portfolio weights and multipliers.

Variable	Value	Description
x_1	0.253661	Weight – Ecopetrol
x_2	0.365849	Weight – ISA
x_3	0.380491	Weight – Bancolombia
λ_1	0.096020	Multiplier – return constraint
λ_2	0.003178	Multiplier – budget constraint

Constraint check:

$$x_1 + x_2 + x_3 = 0.253661 + 0.365849 + 0.380491 = 1.000001 \approx 1 \checkmark$$
$$0.15 \cdot x_1 + 0.12 \cdot x_2 + 0.10 \cdot x_3 = 0.120000 = R \checkmark$$

Method analysis

The solution satisfies both constraints with negligible round-off error. **Financial reading:** to reach a 12% expected return with minimum variance, the investor should hold:

- About 25.4% in Ecopetrol, the asset with the highest return and highest variance.
- About 36.6% in ISA.
- About 38.0% in Bancolombia.

The high weight on Bancolombia happens because, within the feasible set, its covariance with ISA is relatively low. This makes it possible to lower the portfolio variance without missing the return target. The multiplier $\lambda_1 \approx 0.096$ has a KKT meaning: it is the *marginal reduction in variance* obtainable by relaxing the return target by one unit. In economic terms, that is the implicit "cost" of the return constraint.

Numerical check: with different assets and parameters

Using the same method with three different assets (Grupo Sura, Cementos Argos, Nutresa), returns $\mu = (16\%, 13\%, 11\%)$, target $R = 13\%$, and a different covariance matrix, the weights are $x_1 \approx 26.5\%$, $x_2 \approx 33.9\%$, $x_3 \approx 39.7\%$, satisfying both constraints exactly. The procedure works the same way on a different setup.

3.5 Gaussian Elimination with Full Pivoting

Theory

Full pivoting searches, before each elimination step k, for the entry of largest absolute value in the *entire remaining submatrix* (rows and columns $\geq k$) and moves it to the pivot position (k, k) via simultaneous row and column swaps. A permutation vector tracks the column reordering, so that the solution can be put back in the right order at the end.

Theorem: Numerical stability with full pivoting

Gaussian elimination with full pivoting satisfies Wilkinson's growth bound: the entries of the resulting upper triangular matrix U do not grow beyond a moderate polynomial factor of the maximum entry of A. This guarantees numerical stability for any non-singular matrix, including matrices close to singular.

Without pivoting, round-off errors can amplify exponentially. Full pivoting is the most stable variant, although also the most expensive (the pivot search is $\mathcal{O}(n^2)$ per step). In practice, partial pivoting (rows only) is usually enough.

Why it matters here. In the Markowitz KKT matrix, entries of Σ are small (~ 0.01), while the constraint entries are of order 1. This range mismatch can hurt precision without pivoting; full pivoting mitigates the risk.

Results

Full pivoting produces the same portfolio weights as simple elimination:

$$x_1 = 0.253661, \quad x_2 = 0.365849, \quad x_3 = 0.380491$$

Method analysis

The match with simple elimination suggests the problem matrix is numerically well-conditioned at size 5×5. A more rigorous check would compute the condition number $\kappa(A) = \|A\| \cdot \|A^{-1}\|$. In the verification with alternative data, $\kappa(A) \approx 108$, which is moderate. Not ill-conditioned, but not trivial either. For much larger systems (hundreds or thousands of assets) or for Σ close to singular (highly correlated assets), full pivoting becomes essential to avoid losing precision from cancellation.

3.6 Thomas algorithm for tridiagonal systems (additional method)

Theory

A tridiagonal system has the form:

$$\begin{pmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \cdots & 0 \\ 0 & a_3 & b_3 & \ddots & \vdots \\ \vdots & & \ddots & \ddots & c_{n-1} \\ 0 & \cdots & 0 & a_n & b_n \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \\ d_n \end{pmatrix}$$

The **Thomas algorithm** exploits this structure to solve the system in $\mathcal{O}(n)$ operations, compared with $\mathcal{O}(n^3)$ for standard Gaussian elimination.

Forward sweep (eliminating the subdiagonal):

$$b_i \leftarrow b_i - \frac{a_i}{b_{i-1}} c_{i-1}, \qquad d_i \leftarrow d_i - \frac{a_i}{b_{i-1}} d_{i-1}, \quad i = 2, \ldots, n$$

Back substitution:

$$x_n = \frac{d_n}{b_n}, \qquad x_i = \frac{d_i - c_i x_{i+1}}{b_i}, \quad i = n-1, \ldots, 1$$

Theorem: Stability of the Thomas algorithm

If the tridiagonal matrix is *strictly diagonally dominant* ($|b_i| > |a_i| + |c_i|$ for all i), then the Thomas algorithm is numerically stable and requires no pivoting.

Here, $|b_i| = 50$ and $|a_i| + |c_i| = 50$ (except at the endpoints, where it is 25), so weak diagonal dominance holds. The algorithm works without trouble.

Results – bond sensitivity system

Input: $\mathbf{b} = (50, 50, 50, 50)$, $\mathbf{a} = (0, -25, -25, -25)$, $\mathbf{c} = (-25, -25, -25)$, $\mathbf{d} = (1, 1, 1, 1)$.

Table 11: Thomas vs. analytical solution $u(x) = x(1-x)/2$.

Node	x	Numerical	Analytical
u_1	0.2	0.080000	0.080000
u_2	0.4	0.120000	0.120000
u_3	0.6	0.120000	0.120000
u_4	0.8	0.080000	0.080000

Method analysis

The Thomas algorithm matches the analytical solution *exactly* to the decimal places shown. The reason is that the analytical solution $u(x) = x(1-x)/2$ is a degree-2 polynomial, and the centered finite difference is exact for polynomials up to degree 2. Its truncation error involves the fourth derivative, which is zero here.

Why this matters in practice: real quantitative finance applications like bond pricing through yield curve discretization, or finite-difference methods for option pricing such as the Crank-Nicolson scheme for Black-Scholes, run into tridiagonal systems with n in the hundreds or thousands. The gap between $\mathcal{O}(n)$ (Thomas) and $\mathcal{O}(n^3)$ (general Gauss) is huge: for $n = 1000$, that is 10^3 vs. 10^9 operations, six orders of magnitude apart. That is why Thomas is the standard algorithm for tridiagonal systems coming from financial PDE discretizations.

4 Conclusions

The conclusions below apply to the specific bond and portfolio parameters used in this report and should not be generalized without further validation.

1. For the TES bond from equation (2), the eight root-finding methods converge to a YTM of about 8.26%. This exceeds the 7% coupon rate, so the bond trades at a discount for these parameters. A second dataset ($C = \$80$, $n = 7$, $P = \$920$, YTM $\approx 9.62\%$) gave a consistent result.

2. Newton-Raphson shows the predicted quadratic convergence: the error drops from 10^{-3} to 10^{-5} to 10^{-10} in only three steps. The theorem was checked using the ratio $E_{n+1}/E_n^2 \to$ constant. Steffensen reaches the same convergence order without needing derivatives, but it requires a contractive $g(x)$, which is not always available (as in the TES case).

3. Banach's theorem was verified by evaluating $|g'(x)|$ on $[1, 2]$ for $g(x) = (x + 2)^{1/3}$. The maximum was $k \approx 0.1603 < 1$, which guarantees both contraction and convergence.

4. Bisection (19 iter.) and Trisection (13 iter.) are the most robust methods, with guaranteed convergence from a sign change alone. The lower iteration count of trisection does not mean lower total cost: each step needs two evaluations of f, totalling 26 vs. 19 for bisection.

5. Both Gauss variants applied to the KKT system give identical weights for this numerically stable problem (a rigorous check of conditioning, $\kappa(A) \approx 108$ in the alternative-data verification, confirms it is moderately well-conditioned). The minimum-variance portfolio for a 12% return target assigns roughly 25.4% to Ecopetrol, 36.6% to ISA, and 38.0% to Bancolombia.

6. The Thomas algorithm reproduces the analytical solution of the bond sensitivity problem to all decimal places shown, in $\mathcal{O}(n)$ operations. The advantage grows with n, which justifies using the algorithm in any large-scale financial discretization.